

Aprendizado de Máquina Supervisionado –IMD3002

Aula 25 – Convolutional Neural Networks (CNN)

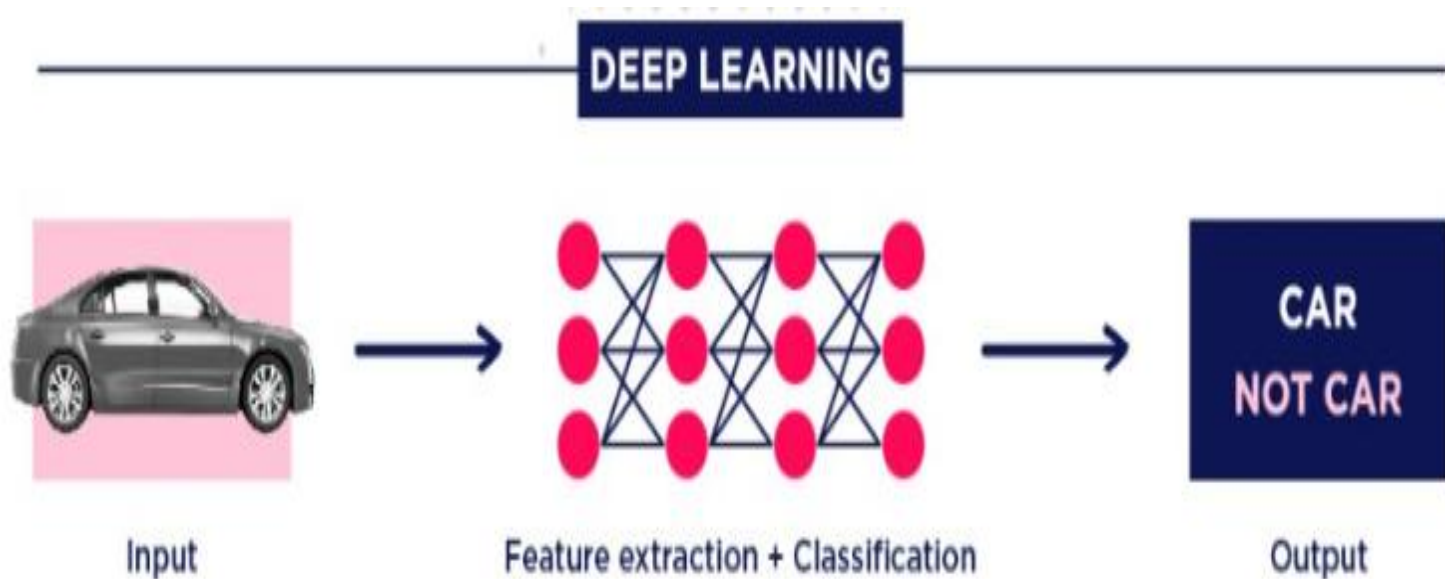
João Carlos Xavier Júnior

jcxavier@imd.ufrn.br

O que é Deep Learning?

- ❑ O **Deep Learning** é um subconjunto do Aprendizado de Máquina que usa Redes Neurais de **várias camadas**.
- ❑ Chamadas de Redes Profundas.
- ❑ Tentam simular o complexo poder de tomada de decisão do cérebro humano.
- ❑ As redes profundas estão incorporadas na maioria das aplicações de **Inteligência Artificial** (IA) em nossas vidas atualmente.

Deep Learning



Modelos de Deep Learning

■ Aplicações

- ❑ Processamento de Imagens
- ❑ Detecção de Objetos em um vídeo
- ❑ Processamento de Linguagem Natural
- ❑ Modelos de tradução de Línguas
- ❑ Modelos de Línguas

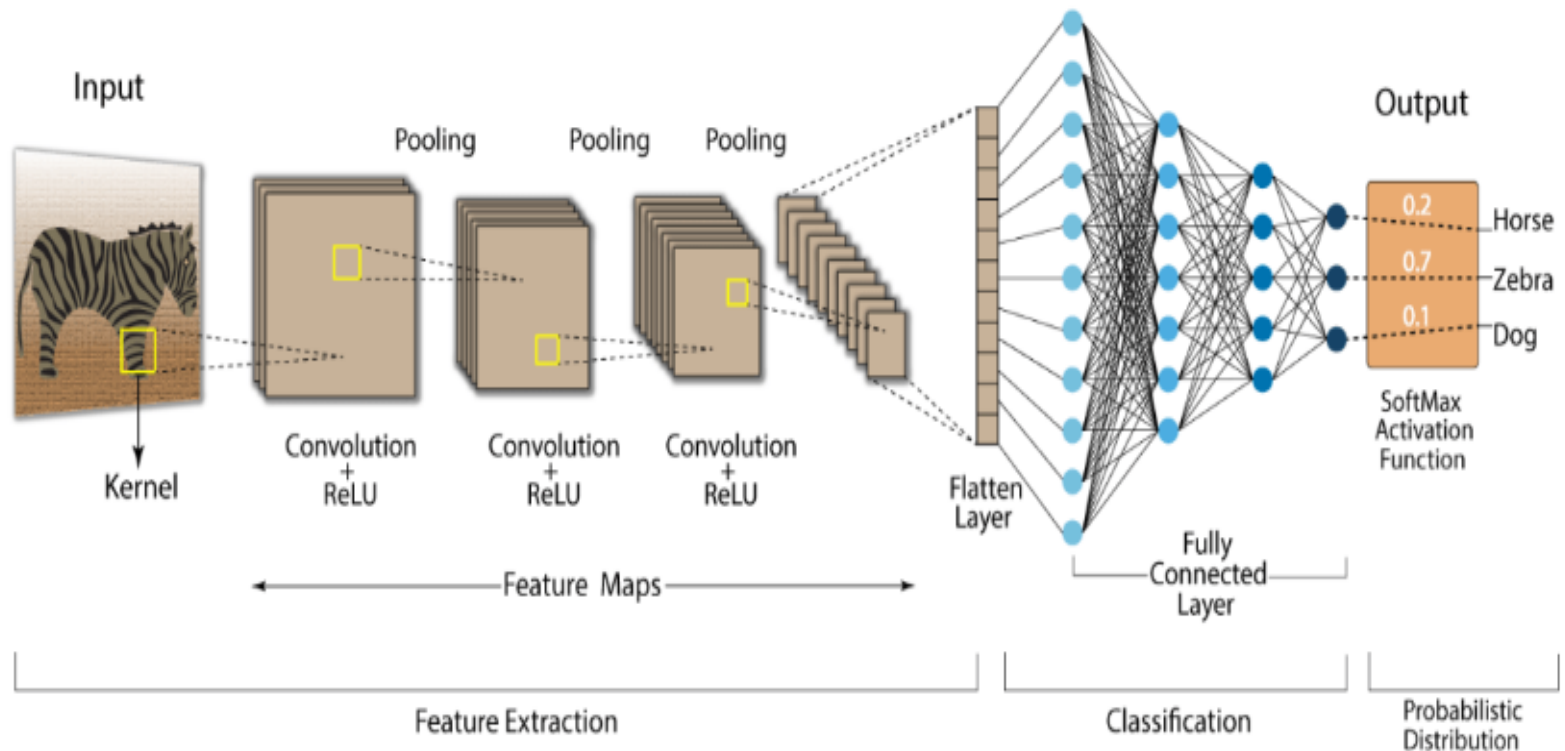
■ Modelos

- ❑ Arquiteturas CNNs
- ❑ YOLO (You Only Look Once)
- ❑ RNNs (Rede Neural Recorrente), LSTM (Long Short-Term Memory)
- ❑ LSTM e GRUs (Gated Recurrent Unit)
- ❑ Transformers e LLMs (Large Language Models)

Rede Neural Convolutacional

Rede Neural Convolucional

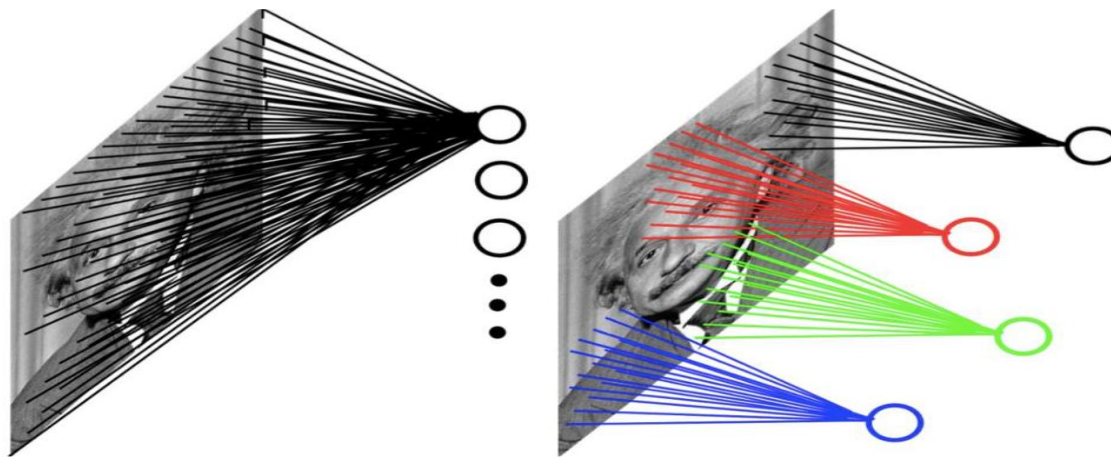
□ Arquitetura:



Rede Neural Convolucional

❑ Campos Receptivos Locais (*Local Receptive Fields*):

- ❖ Objetivo: diminuir o número de parâmetros.
- ❖ Exemplo: 200x200 imagem
 - MLP: 40.000 hidden units => 1,6 bilhões de parâmetros.
 - CNN: 5x5 kernel, 100 campos receptivos=> 2.500 parâmetros.



Camada de Convolução

❑ Operação de convolução:

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

imagem

1	0	1
0	1	0
1	0	1

operador de convolução
(filtro, kernel)

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

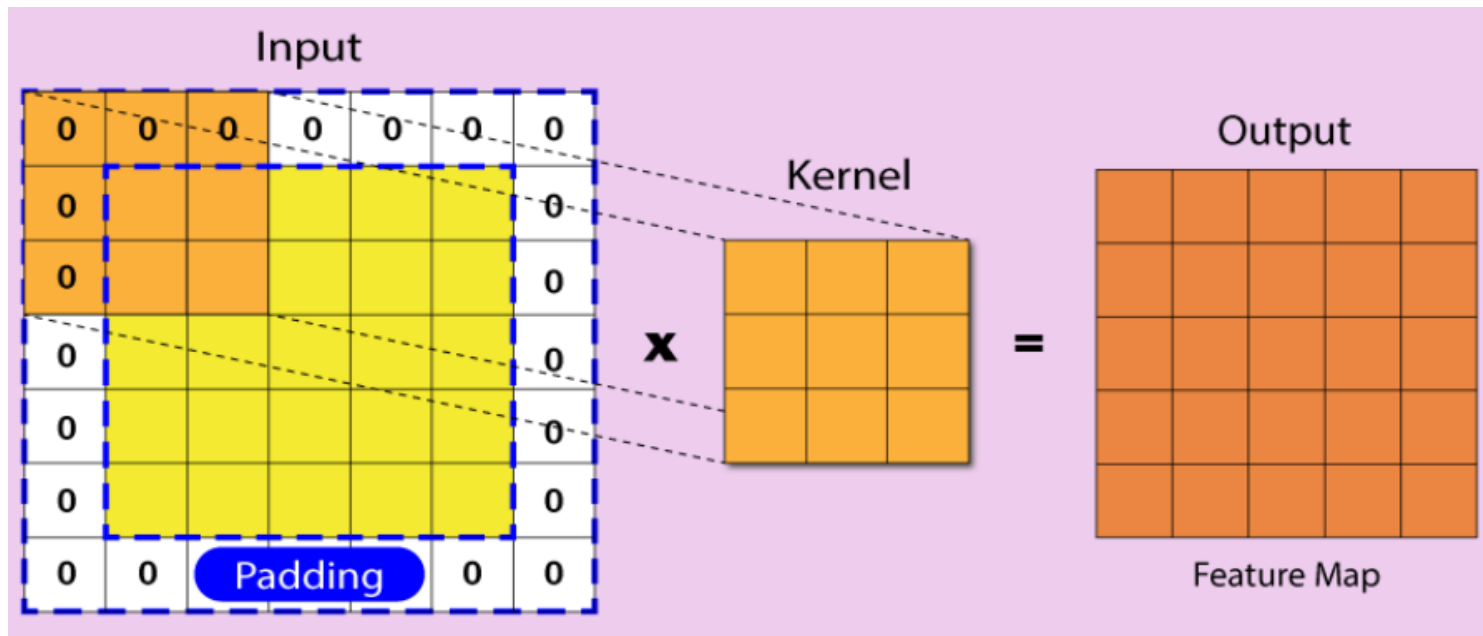
Image

4		

Convolved
Feature

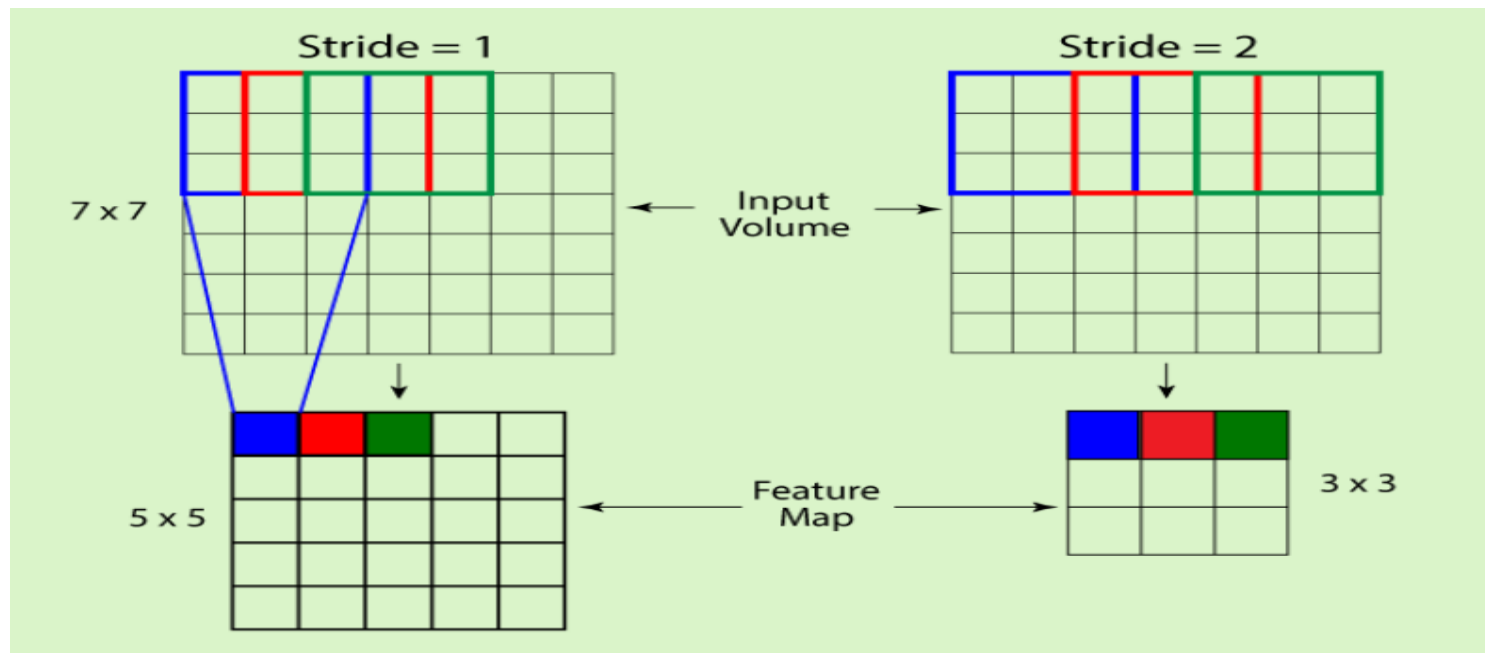
Mapa de Características

- ❑ Zero-padding: adição simetricamente zeros a cada fatia (*depth slice*) do volume de entrada.



Mapa de Características

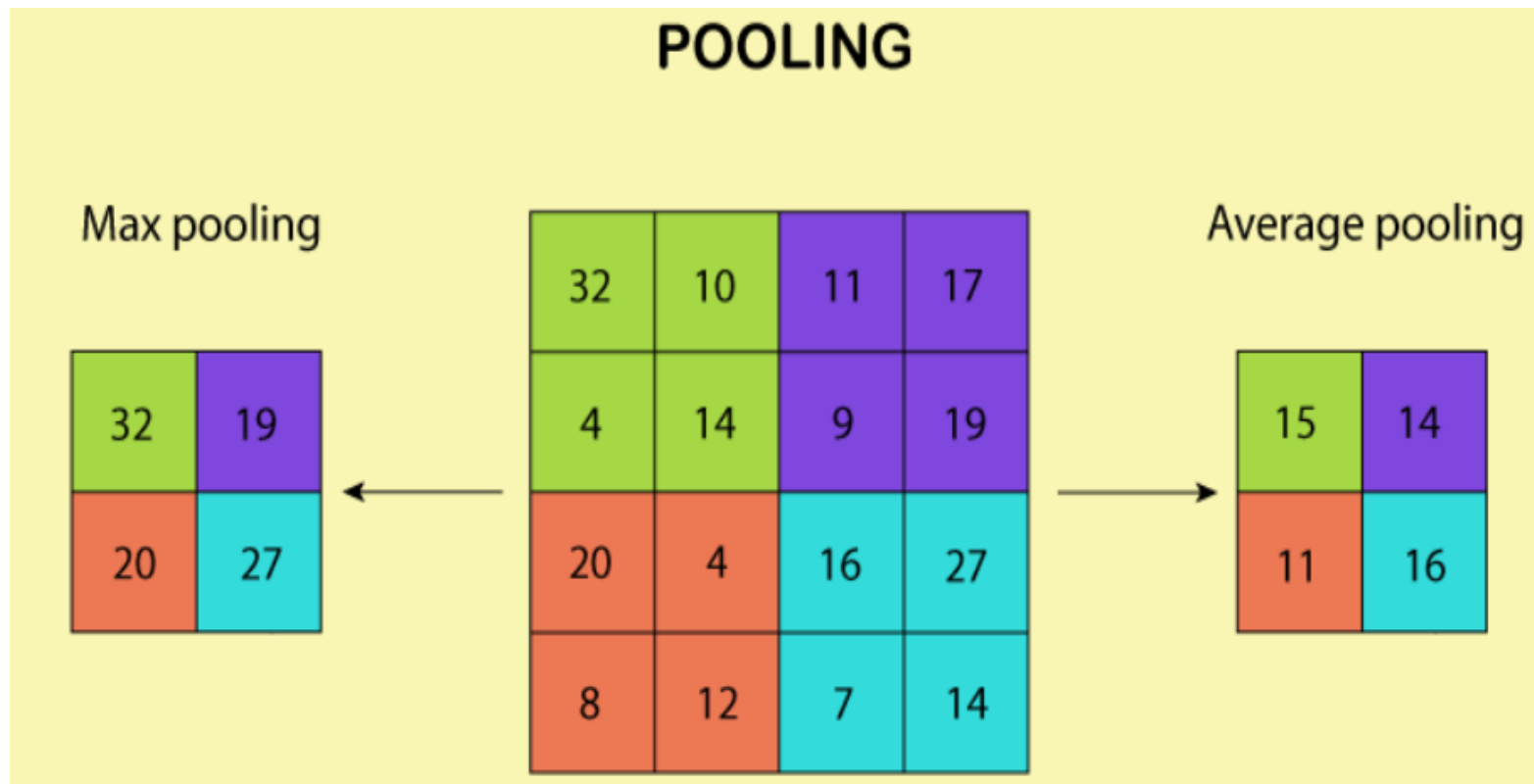
- ❑ O passo (*stride*, S) é um hiperparâmetro que controla como o filtro é “deslizado” pelo volume de entrada durante a convolução.



Pooling

- ❑ Técnica usada para **reduzir a dimensionalidade espacial** (largura e altura) dos mapas de características gerados pelas camadas de convolução.
- ❑ Isso resulta em uma representação mais concisa e gerenciável, diminuindo o número de parâmetros e o processamento na rede.

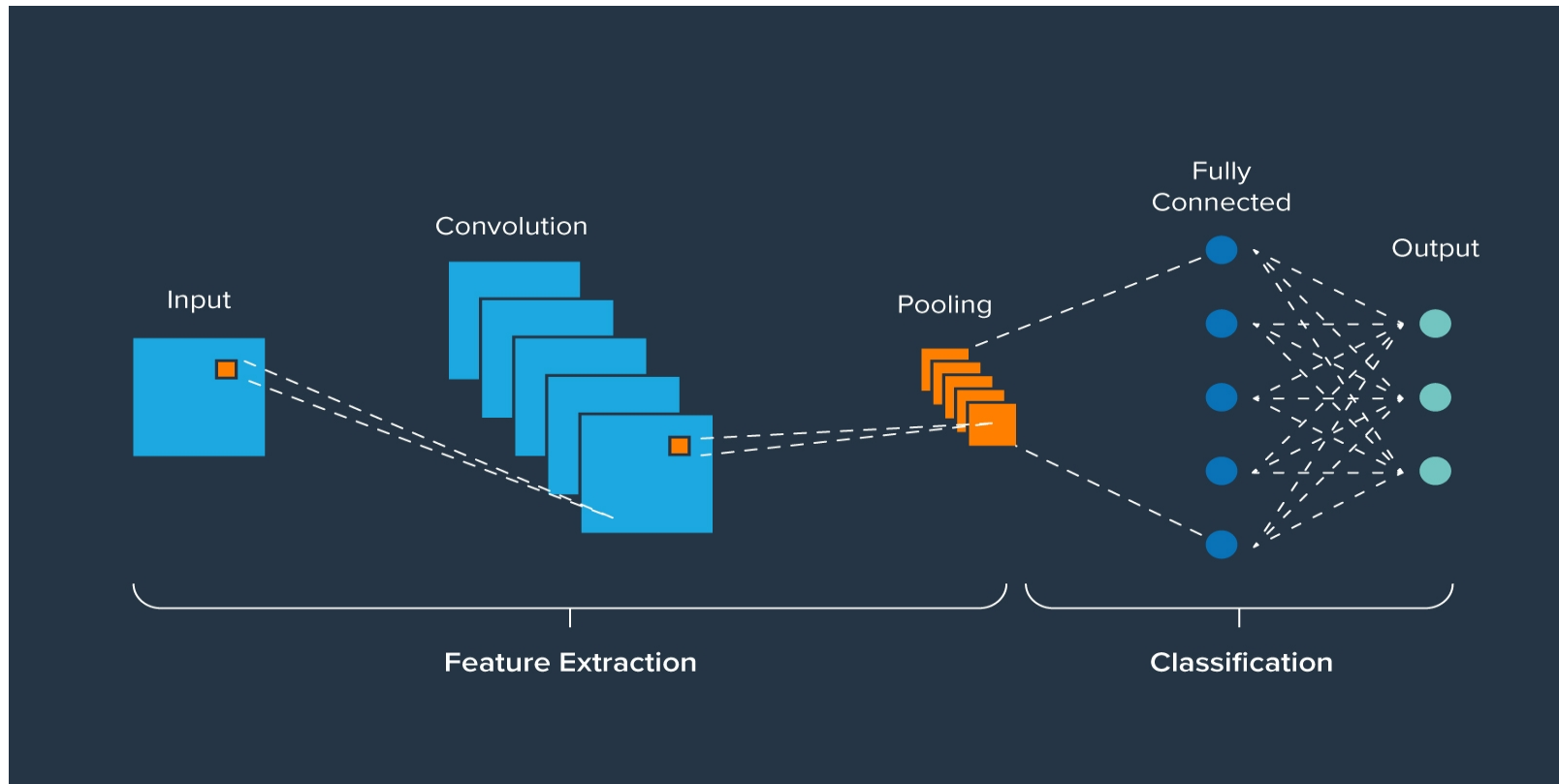
Pooling



Classificação de Imagens com CNN

Classificação de Imagem

□ Arquitetura de uma CNN:



<https://www.exxactcorp.com/blog/Deep-Learning/Image-Classification-with-DCNNs>

Classificação de Imagem

Imagens:

Meu Drive > IMD3202 > AM_Aula25 ▾

Tipo ▾ Pessoas ▾ Modificado ▾ Fonte ▾

Nome	Proprietário
 Notebooks	 eu
 Imagens	 eu

... > AM_Aula25 > Imagens ▾

Tipo ▾ Pessoas ▾ Modificado ▾ Fonte ▾

Nome	Proprietário
 0	 eu
 1	 eu
 2	 eu

Classificação de Imagem

📁 Notebook:

... > AM_Aula25 > Notebooks ▾ 👤

Tipo ▾ Pessoas ▾ Modificado ▾ Fonte ▾

Nome

Proprietário



CNN_Model_Testes.ipynb 👤



eu



Treinamento_CNN.ipynb 👤

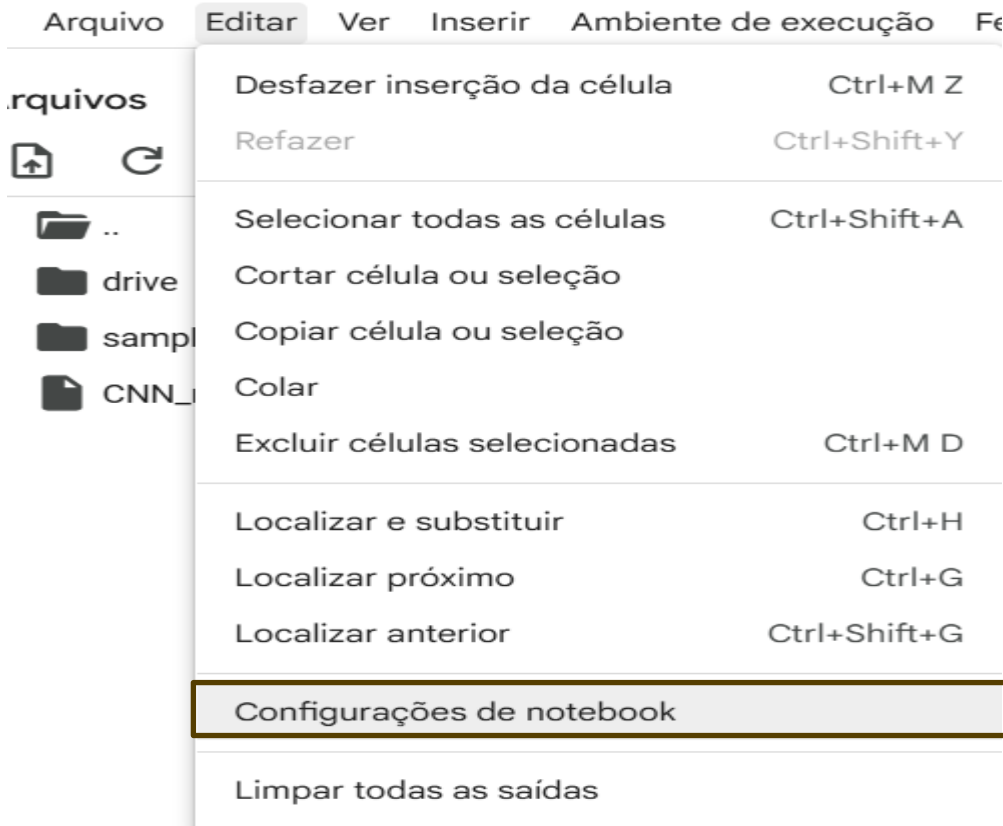


eu

<https://drive.google.com/drive/folders/1jlUBj4H1n8YiRHIX8lDrQqeUSxLe3bUz?usp=sharing>

Classificação de Imagem

❑ Configurando o uso de GPU:



Classificação de Imagem

❏ Configurando o uso de GPU:

Configurações de notebook

Tipo de ambiente de execução

Python 3 ▼

Acelerador de hardware

☐ CPU ☒ GPUs: T4 ☐ GPUs: A100 ☐ GPUs: L4 ☐ TPU v2-8
☐ TPU v5e1

Quer acesso a GPUs premium? [Compre mais unidades de computação](#)

Sua organização pode comprar licenças do Colab Pro e do Colab Pro+ em massa. [Saiba mais](#)

☐ Executar automaticamente a primeira célula ou seção em qualquer execução

☐ Omitir saída da célula de código ao salvar este notebook

Cancelar

Salvar

Classificação de Imagem

❑ Configurando o uso de GPU:

Como treinar um modelo de Deep Learning no Google Colab?



Pierre Guillou · Follow

5 min read · Feb 22, 2021



https://medium.com/@pierre_guillou/como-treinar-um-modelo-de-deep-learning-no-colab-e3cc4da64d9a

Classificação de Imagem

❑ Configurando o uso de GPU:

✓ Configurando o uso de GPU

```
▶ import tensorflow as tf  
dispositivo = tf.test.gpu_device_name()  
print(dispositivo)
```

```
⇒ /device:GPU:0
```

Classificação de Imagem

❑ Carregando e Transformando as Imagens:

Etapa 1 - Configurando Google Drive

```
from google.colab import drive  
drive.mount('/content/drive')
```

```
path = "/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens"
```

```
Mounted at /content/drive
```

Classificação de Imagem

❏ Carregando e Transformando as Imagens:

Etapa 2 - Processando as Imagens

```
import numpy as np # type: ignore
import os
import matplotlib.pyplot as plt # type: ignore

## Carregar as Libs necessarias
from keras.preprocessing.image import load_img # type: ignore

### Parâmetros
imageDimesions = (128,128,3)

## Importar Imagens
count = 0
images = []
classNo = []
pastas = os.listdir(path)
print("Total de Classes:",len(pastas))
noOfClasses=len(pastas)

for pt in range (0,len(pastas)):
    arquivos = os.listdir(path+"/"+str(count))
    for arq in arquivos:
        image = load_img(path+"/"+str(count)+"/"+arq, target_size=(128, 128))
        print(path+"/"+str(count)+"/"+arq)
        images.append(image)
        classNo.append(count)

    count +=1

images = np.array(images)
classNo = np.array(classNo)

print(images)
print(classNo)
```

Classificação de Imagem

❑ Carregando e Transformando as Imagens:

```
➡ Total de Classes: 3
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_89.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_174.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_59.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_121.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_189.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_137.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_77.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_187.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_148.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_71.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_217.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_216.jpg

/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_150.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_202.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/0/american_bulldog_8.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_55.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_175.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_106.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_1.jpg

/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_111.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_74.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_47.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/1/Bengal_197.jpg
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/2/01_001.png
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/2/01_003.png
/content/drive/MyDrive/IMD3202/AM_Aula25/Imagens/2/01_002.png
```

Classificação de Imagem

❏ Separando as Imagens:

Etapa 3 - Separando as Imagens

```
▶ ## Separando Imagens
from sklearn.model_selection import train_test_split # type: ignore
from keras import utils

X_train, X_test, y_train, y_test = train_test_split(images, classNo, test_size=0.3)
X_train, X_validation, y_train, y_validation = train_test_split(X_train, y_train, test_size=0.3)

### Separando as imagens e seus rótulos
y_train = utils.to_categorical(y_train,noOfClasses)
y_validation = utils.to_categorical(y_validation,noOfClasses)
y_test = utils.to_categorical(y_test,noOfClasses)

# Print out shapes
print("X train:", X_train.shape)
print("y train:", y_train.shape)
print("X val:", X_validation.shape)
print("y val:", y_validation.shape)
print("X test:", X_test.shape)
print("y test:", y_test.shape)
```

```
⇒ X train: (524, 128, 128, 3)
   y train: (524, 3)
   X val: (225, 128, 128, 3)
   y val: (225, 3)
   X test: (321, 128, 128, 3)
   y test: (321, 3)
```

Classificação de Imagem

□ Arquitetura da CNN:

Arquitetura da Rede e Funções de Ativação

A rede começa com camadas convolucionais. Cada camada convolucional aplica um conjunto de filtros para extrair características da imagem, sendo essencial para aprender padrões espaciais hierárquicos. Após cada camada convolucional, utilizamos:

- Normalização em Lote: Ajuda a estabilizar e acelerar o treinamento, normalizando as ativações da camada anterior.
- ReLU (Rectified Linear Unit): Uma função de ativação que introduz não-linearidades no modelo, permitindo que a rede aprenda representações complexas. ReLU é preferida por sua eficiência computacional e porque reduz o problema do desaparecimento do gradiente.
- Pooling Máximo: Reduz a dimensionalidade espacial dos dados, ajudando a tornar a representação mais abstrata e reduzindo o risco de overfitting.
- Dropout: Uma técnica de regularização que “desliga” aleatoriamente alguns neurônios durante o treinamento, o que ajuda a prevenir o overfitting.

<https://betinioheleno.medium.com/implementando-uma-cnn-para-classifica%C3%A7%C3%A3o-de-g%C3%AAnero-com-tensorflow-e-keras-uma-abordagem-detalhada-77aa9cdfdec9>

Classificação de Imagem

Arquitetura da CNN:

Etapa 4 - Montando a Arquitetura da Rede

```
▶ from keras.models import Sequential # Type of model we wish to use
   from keras.layers import Dense, Dropout, Flatten, Activation, Conv2D, MaxPooling2D # Types of layers we wish to use

# A sequential model is a linear stack of layers
model = Sequential()

model.add(Conv2D(32, kernel_size=(3,3), input_shape=imageDimensions, activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.1))

model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(noOfClasses, activation='softmax'))

# COMPILE MODEL
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['acc'])

# Print out model summary
print(model.summary())
```

Classificação de Imagem

□ Arquitetura da CNN:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
dropout (Dropout)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	36,928
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 128)	1,605,760
dense_1 (Dense)	(None, 3)	387

Total params: 1,662,467 (6.34 MB)

Trainable params: 1,662,467 (6.34 MB)

Non-trainable params: 0 (0.00 B)

Classificação de Imagem

❏ Treinando a CNN:

```
[64] ### Train the model
      history = model.fit(X_train,
                          y_train,
                          batch_size=32,
                          epochs=100,
                          verbose=1,
                          validation_data=(X_validation, y_validation))
```

```
print('Fim do treinamento')
```

```
⇒ 11/11 ————— 0s 18ms/step - acc: 1.0000 - loss: 8.2666e-06 - val_acc: 0.7778 - val_loss: 2.7756
   Epoch 73/100
   11/11 ————— 0s 20ms/step - acc: 1.0000 - loss: 1.3818e-05 - val_acc: 0.7778 - val_loss: 2.7732
   Epoch 74/100
   11/11 ————— 0s 20ms/step - acc: 1.0000 - loss: 1.3458e-05 - val_acc: 0.7778 - val_loss: 2.7735
   Epoch 75/100
   11/11 ————— 0s 19ms/step - acc: 1.0000 - loss: 1.1536e-05 - val_acc: 0.7778 - val_loss: 2.7744
   Epoch 76/100
   11/11 ————— 0s 20ms/step - acc: 1.0000 - loss: 6.9333e-06 - val_acc: 0.7778 - val_loss: 2.7784
   Epoch 77/100
   11/11 ————— 0s 19ms/step - acc: 1.0000 - loss: 9.9975e-06 - val_acc: 0.7778 - val_loss: 2.7955
   Epoch 78/100
   11/11 ————— 0s 22ms/step - acc: 1.0000 - loss: 5.2960e-06 - val_acc: 0.7778 - val_loss: 2.8118
   Epoch 79/100
   11/11 ————— 0s 20ms/step - acc: 1.0000 - loss: 2.9967e-05 - val_acc: 0.7778 - val_loss: 2.8145
   Epoch 80/100
   11/11 ————— 0s 16ms/step - acc: 1.0000 - loss: 5.6109e-06 - val_acc: 0.7778 - val_loss: 2.8189
```

Classificação de Imagem

❑ Avaliando o modelo:

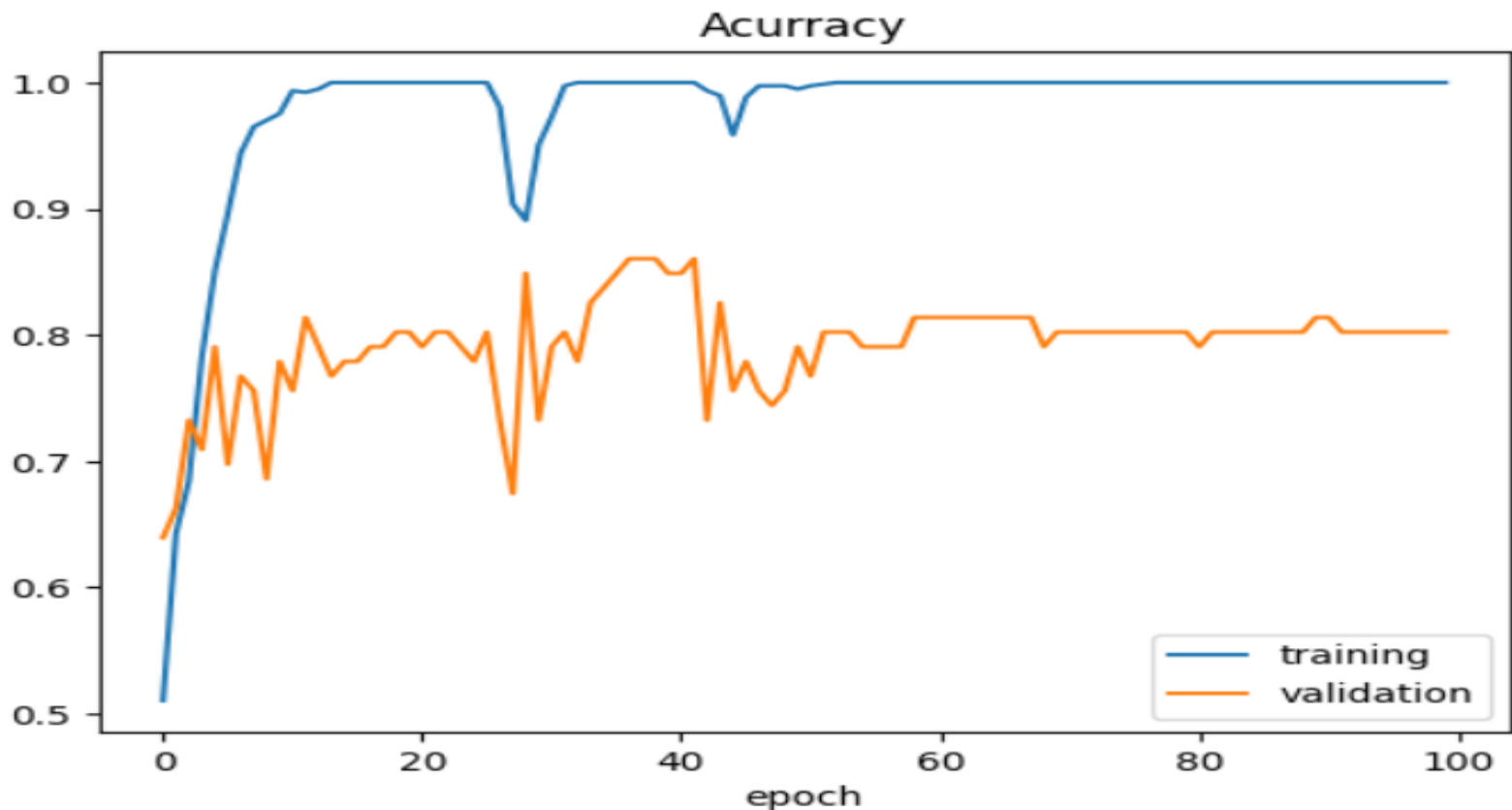
```
[66] ## Mostrar histórico de treinamento
plt.figure
plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.legend(['training', 'validation'])
plt.title('Acurracy')
plt.xlabel('epoch')
plt.show()

score = model.evaluate(X_test, y_test, verbose=0)
print('Test Score:', score[0])
print('Test Accuracy:', score[1])

## Salvar modelo
model.save('CNN_model.h5')
print('Modelo Salvo!')
```

Classificação de Imagem

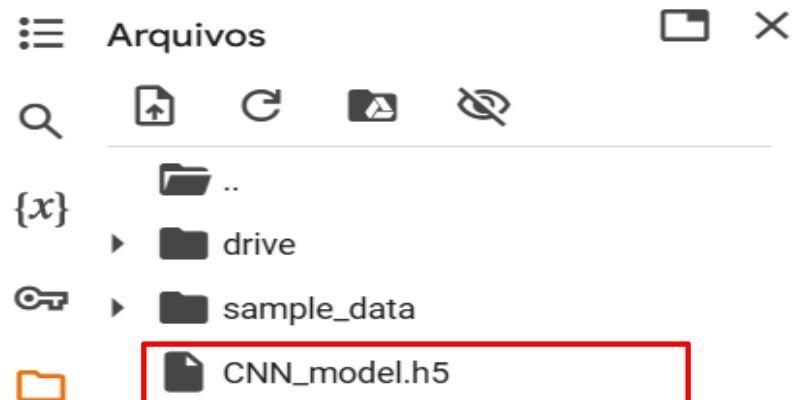
❑ Avaliando o modelo:



Classificação de Imagem

❑ Avaliando o modelo:

```
WARNING:absl:You are saving your model as an HDF5  
Test Score: 2.3049745559692383  
Test Accuracy: 0.7850467562675476  
Modelo Salvo!
```



Utilizando o Modelo Salvo

Modelo de ML

❑ Abrindo uma imagem de teste:

```
▶ from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np
from PIL import Image

image_path = '/content/Cat_teste_01.jpg'
#image_path = '/content/Dog_teste_01.jpg'
#image_path = '/content/Horse_teste_01.jpg'

image = Image.open(image_path)
image = image.resize((128, 128))
image
```



Modelo de ML

❑ Utilizando o modelo:

```
from keras.models import load_model
model = load_model('/content/CNN_model.h5')

classes = ["dog", "cat", "horse"]
image = img_to_array(image)
image = np.expand_dims(image, axis=0)

predictions = model.predict(image)
predictions
predicted_class = np.argmax(predictions, axis=1)

### Predição da classe
print('Classe: ', classes[predicted_class[0]])

### Probabilidade de predição
probabilityValue = np.amax(predictions)
print('Probabilidade', probabilityValue)
```

Modelo de ML

❑ Utilizando o modelo:

1/1  0s 119ms/step

Classe: horse

Probabilidade 0.8516578



Modelo de ML

❑ Abrindo outra imagem de teste:

```
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np
from PIL import Image

#image_path = '/content/Cat_teste_01.jpg'
image_path = '/content/Dog_teste_01.jpg'
#image_path = '/content/Horse_teste_01.jpg'

image = Image.open(image_path)
image = image.resize((128, 128))
image
```



Modelo de ML

❑ Utilizando o modelo (again):

1/1  0s 114ms/step

Classe: dog

Probabilidade 0.9937688



Modelo de ML

❑ Utilizando o modelo (again):

```
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np
from PIL import Image
```

```
#image_path = '/content/Cat_teste_01.jpg'
#image_path = '/content/Dog_teste_01.jpg'
image_path = '/content/Horse_teste_01.jpg'
```

```
image = Image.open(image_path)
image = image.resize((128, 128))
image
```



Modelo de ML

❑ Utilizando o modelo (again):

1/1  0s 119ms/step

Classe: dog

Probabilidade 0.9987218



Dúvidas ...



Obrigado!!!

